

Getting Started

Many factors have contributed to Clojure's quick rise. A quick web search will likely tell you that Clojure:

- is a functional language,
- is a Lisp for the JVM, and
- has special features for dealing with concurrency.

All of these things are important, but none of them is the key to thinking in Clojure. In our opinion, there are two key concepts that drive everything else in Clojure: simplicity and power.

Simplicity has several meanings that are relevant in software, but the definition we mean is the original and best one: a thing is simple if it is not compound. Simple components allow systems to do what their designers intend, without also doing other things irrelevant to the task at hand. In our experience, irrelevant complexity quickly becomes dangerous complexity.

Power also has many meanings. The one we care about here is sufficiency to the tasks we want to undertake. To feel powerful as a programmer, you need to build on a substrate that is itself capable and widely deployed, e.g., the JVM. Then, your tools must give you full, unrestricted access to that power. Power is often a gatekeeping requirement for projects that must get the most out of their platform.

As programmers, we have spent years tolerating baroquely complex tools that were the only way to get the power we needed or accepting reduced power for a sanity-enhancing simplification of the programming model. Some trade-offs are truly fundamental, but power vs. simplicity is not one of them. Clojure shows that power and simplicity can go hand in hand.

1.1 Why Clojure?

All of the distinctive features in Clojure are there to provide simplicity, power, or both. Here are a few examples:

- Functional programming is simple, in that it isolates calculation from state and identity. Benefits: functional programs are easier to understand, write, test, optimize, and parallelize.
- Clojure’s Java interop forms are powerful, giving you direct access to the semantics of the Java platform. Benefits: you can have performance and semantic equivalence to Java. Most importantly, you will never need to “drop down” to a lower-level language for a little extra power.
- Lisp is simple in two critical ways: it separates reading from evaluation, and the language syntax is made from a tiny number of orthogonal parts. Benefits: syntactic abstraction captures design patterns, and S-expressions are XML, JSON, and SQL as they should have been.
- Lisp is also powerful, providing a compiler and macro system at runtime. Benefits: Lisp has late-bound decision making and easy DSLs.
- Clojure’s time model is simple, separating values, identities, state, and time. Benefits: programs can perceive and remember information, without fear that somebody is about to scribble over the past.
- Protocols are simple, separating polymorphism from derivation. Benefits: you get safe, ad hoc extensibility of type and abstractions, without a tangle of design patterns or fragile monkey patching.

This list of features acts as a road map for the rest of the book, so don’t worry if you don’t follow every little detail here. Each feature gets an entire chapter later.

Let’s see some of these features in action by building a small application. Along the way, you will learn how to load and execute the larger examples we will use later in the book.

Clojure Is Elegant

Clojure is high-signal, low-noise. As a result, Clojure programs are short programs. Short programs are cheaper to build, cheaper to deploy, and cheaper to maintain.¹ This is particularly true when the programs are concise

1. *Software Estimation: Demystifying the Black Art* [McC06] is a great read and makes the case that smaller is cheaper.

rather than merely terse. As an example, consider the following Java code, from Apache Commons:

data/snippets/isBlank.java

```
public class StringUtils {
    public static boolean isBlank(String str) {
        int strLen;
        if (str == null || (strLen = str.length()) == 0) {
            return true;
        }
        for (int i = 0; i < strLen; i++) {
            if ((Character.isWhitespace(str.charAt(i)) == false)) {
                return false;
            }
        }
        return true;
    }
}
```

The `isBlank()` method checks to see whether a string is *blank*: either empty or consisting of only whitespace. Here is a similar implementation in Clojure:

src/examples/introduction.clj

```
(defn blank? [str]
  (every? #(Character/isWhitespace %) str))
```

The Clojure version is shorter. But even more important, it is *simpler*: it has no variables, no mutable state, and no branches. This is possible thanks to *higher-order functions*. A higher-order function is a function that takes functions as arguments and/or returns functions as results. The `every?` function takes a function and a collection as its arguments and returns true if that function returns true for every item in the collection.

Because the Clojure version has no branches, it is easier to read and test. These benefits are magnified in larger programs. Also, while the code is concise, it is still readable. In fact, the Clojure program reads like a *definition* of blank: a string is blank if every character in it is whitespace. This is much better than the Commons method, which hides the definition of blank behind the implementation detail of loops and if statements.

As another example, consider defining a trivial Person class in Java:

data/snippets/Person.java

```
public class Person {
    private String firstName;
    private String lastName;

    public Person(String firstName, String lastName) {
        this.firstName = firstName;
    }
}
```

```

        this.lastName = lastName;
    }

    public String getFirstName() {
        return firstName;
    }

    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }

    public String getLastName() {
        return lastName;
    }

    public void setLastName(String lastName) {
        this.lastName = lastName;
    }
}

```

In Clojure, you would define `Person` with a single line:

```
(defrecord Person [first-name last-name])
```

and work with the record like so:

```

(def foo (->Person "Aaron" "Bedra"))
-> #'user/foo
foo
-> #:user.Person{:first-name "Aaron", :last-name "Bedra"}

```

`defrecord` and related functions are covered in [Section 6.3, *Protocols*, on page 147](#).

Other than being an order of magnitude shorter, the Clojure approach differs in that a Clojure `Person` is *immutable*. Immutable data structures are naturally thread safe, and update capabilities can be layered when using Clojure's references, agents, and atoms, which are covered in [Chapter 5, *State*, on page 113](#). Because records are immutable, Clojure also provides correct implementations of `hashCode()` and `equals()` automatically.

Clojure has a lot of elegance baked in, but if you find something missing, you can add it yourself, thanks to the power of Lisp.

Clojure Is Lisp Reloaded

Clojure is a Lisp. For decades, Lisp advocates have pointed out the advantages that Lisp has over, well, everything else. At the same time, Lisp's world domination plan seems to be proceeding slowly.

Like any other Lisp, Clojure faces two challenges:

- Clojure must succeed as a Lisp by persuading Lisp programmers that Clojure embraces the critical parts of Lisp.
- At the same time, Clojure needs to succeed *where past Lisps have failed* by winning support from the broader community of programmers.

Clojure meets these challenges by providing the metaprogramming capabilities of Lisp and at the same time embracing a set of syntax enhancements that make Clojure friendlier to non-Lisp programmers.

Why Lisp?

Lisps have a tiny language core, almost no syntax, and a powerful macro facility. With these features, you can bend Lisp to meet your design, instead of the other way around. By contrast, consider the following snippet of Java code:

```
public class Person {
    private String firstName;
    public String getFirstName() {
        // continues
    }
}
```

In this code, `getFirstName()` is a method. Methods are polymorphic and can bend to meet your needs. But the interpretation of *every other word* in the example is *fixed by the language*. Sometimes you really need to change what these words mean. So, for example, you might do the following:

- Redefine `private` to mean “private for production code but public for serialization and unit tests.”
- Redefine `class` to automatically generate getters and setters for private fields, unless otherwise directed.
- Create a subclass of `class` that provides callback hooks for life-cycle events. For example, a life cycle-aware class could fire an event whenever an instance of the class is created.

We have seen programs that needed all these features. Without them, programmers resort to repetitive, error-prone workarounds. Literally *millions* of lines of code have been written to work around missing features in programming languages.

In most languages, you would have to petition the language implementer to add the kinds of features mentioned earlier. In Clojure, you can add your own language features with *macros* ([Chapter 7, Macros, on page 165](#)). Clojure itself is built out of macros such as `defrecord`:

```
(defrecord name [arg1 arg2 arg3])
```

If you need different semantics, write your own macro. If you want a variant of records with strong typing and configurable null-checking for all fields, you can create your own `defrecord` macro, to be used like this:

```
(defrecord name [Type :arg1 Type :arg2 Type :arg3]
  :allow-nulls false)
```

This ability to reprogram the language from within the language is the unique advantage of Lisp. You will see facets of this idea described in various ways:

- Lisp is homoiconic.² That is, Lisp code is just Lisp data. This makes it easy for programs to write other programs.
- The whole language is there, all the time. Paul Graham's essay "Revenge of the Nerds"³ explains why this is so powerful.

Lisp syntax also eliminates rules for operator precedence and associativity. You will not find a table documenting operator precedence or associativity anywhere in this book. With fully parenthesized expressions, there is no possible ambiguity.

The downside of Lisp's simple, regular syntax, at least for beginners, is Lisp's fixation on parentheses and on lists as the core datatype. Clojure offers an interesting combination of features that makes Lisp more approachable for non-Lispers.

Lisp, with Fewer Parentheses

Clojure offers significant advantages for programmers coming to it from other Lisps:

- Clojure generalizes Lisp's physical list into an abstraction called a *sequence*. This preserves the power of lists, while extending that power to a variety of other data structures.
- Clojure's reliance on the JVM provides a standard library and a deployment platform with great reach.
- Clojure's approach to symbol resolution and syntax quoting makes it easier to write many common macros.

2. <http://en.wikipedia.org/wiki/Homoiconicity>

3. <http://www.paulgraham.com/icad.html>

Many Clojure programmers will be new to Lisp, and they have probably heard bad things about all those parentheses. Clojure keeps the parentheses (and the power of Lisp!) but improves on traditional Lisp syntax in several ways:

- Clojure provides a convenient literal syntax for a wide variety of data structures besides just lists: regular expressions, maps, sets, vectors, and metadata. These features make Clojure code less “listy” than most Lisps. For example, function parameters are specified in a vector: `[]` instead of a list: `()`.

```
src/examples/introduction.clj
```

```
(defn hello-world [username]
  (println (format "Hello, %s" username)))
```

The vector makes the argument list jump out visually and makes Clojure function definitions easy to read.

- In Clojure, unlike most Lisps, commas are whitespace.

```
; make vectors look like arrays in other languages
[1, 2, 3, 4]
-> [1 2 3 4]
```

- Idiomatic Clojure does not nest parentheses more than necessary. Consider the `cond` macro, present in both Common Lisp and Clojure. `cond` evaluates a set of test/result pairs, returning the first result for which a test form yields true. Each test/result pair is grouped with parentheses, like so:

```
; Common Lisp cond
(cond ((= x 10) "equal")
      (> x 10) "more")
```

Clojure avoids the extra parentheses:

```
; Clojure cond
(cond (= x 10) "equal"
      > x 10) "more")
```

This is an aesthetic decision, and both approaches have their supporters. The important thing is that Clojure takes the opportunity to be less Lispy when it can do so without compromising Lisp’s power.

Clojure is an excellent Lisp, both for Lisp experts and for Lisp beginners.

Clojure Is a Functional Language

Clojure is a functional language but not a pure functional language like Haskell. Functional languages have the following properties:

- Functions are *first-class objects*. That is, functions can be created at runtime, passed around, returned, and in general used like any other datatype.
- Data is immutable.
- Functions are *pure*; that is, they have no side effects.

For many tasks, functional programs are easier to understand, less error-prone, and *much* easier to reuse. For example, the following short program searches a database of compositions for every composer who has written a composition named “Requiem”:

```
(for [c compositions :when (= "Requiem" (:name c))] (:composer c))
-> ("W. A. Mozart" "Giuseppe Verdi")
```

The name `for` does not introduce a loop but a *list comprehension*. Read the earlier code as “For each `c` in `compositions`, where the name of `c` is “Requiem”, yield the composer of `c`.” List comprehension is covered more fully in [Transforming Sequences, on page 66](#).

This example has four desirable properties:

- It is *simple*; it has no loops, variables, or mutable state.
- It is *thread safe*; no locking is needed.
- It is *parallelizable*; you could farm out individual steps to multiple threads without changing the code for each step.
- It is *generic*; compositions could be a plain set or XML or a database result set.

Contrast functional programs with *imperative* programs, where explicit statements alter program state. Most object-oriented programs are written in an imperative style and have *none* of the advantages listed earlier; they are unnecessarily complex, not thread safe, not parallelizable, and difficult to generalize. (For a head-to-head comparison of functional and imperative styles, skip forward to [Section 2.7, Where’s My for Loop?, on page 48](#).)

People have known about the advantages of functional languages for a while now. And yet, pure functional languages like Haskell have not taken over the world, because developers find that not everything fits easily into the pure functional view.

There are four reasons that Clojure can attract more interest now than functional languages have in the past:

- Functional programming is more urgent today than ever before. Massively multicore hardware is right around the corner, and functional languages provide a clear approach for taking advantage of it. Functional programming is covered in [Chapter 4, *Functional Programming*, on page 85](#).
- Purely functional languages can make it awkward to model state that really needs to change. Clojure provides a structured mechanism for working with changeable state via software transactional memory and refs ([on page 115](#)), agents ([on page 123](#)), atoms ([on page 122](#)), and dynamic binding ([on page 127](#)).
- Many functional languages are statically typed. Clojure's dynamic typing makes it more accessible for programmers learning functional programming.
- Clojure's Java invocation approach is *not* functional. When you call Java, you enter the familiar, mutable world. This offers a comfortable haven for beginners learning functional programming and a pragmatic alternative to functional style when you need it. Java invocation is covered in [Chapter 9, *Java Down and Dirty*, on page 203](#).

Clojure's approach to changing state enables concurrency without explicit locking and complements Clojure's functional core.

Clojure Simplifies Concurrent Programming

Clojure's support for functional programming makes it easy to write thread-safe code. Since immutable data structures cannot *ever* change, there is no danger of data corruption based on another thread's activity.

However, Clojure's support for concurrency goes beyond just functional programming. When you need references to mutable data, Clojure protects them via software transactional memory (STM). STM is a higher-level approach to thread safety than the locking mechanisms that Java provides. Rather than creating fragile, error-prone locking strategies, you can protect shared state with transactions. This is much more productive, because many programmers have a good understanding of transactions based on experience with databases.

For example, the following code creates a working, thread-safe, in-memory database of accounts:

```
(def accounts (ref #{}))
(defrecord Account [id balance])
```

The `ref` function creates a transactionally protected reference to the current state of the database. Updating is trivial. The following code adds a new account to the database:

```
(dosync
  (alter accounts conj (->Account "CLJ" 1000.00)))
```

The `dosync` causes the update to accounts to execute inside a transaction. This guarantees thread safety, and it is easier to use than locking. With transactions, you never have to worry about which objects to lock or in what order. The transactional approach will also perform better under some common usage scenarios, because (for example) readers will never block.

Although the example here is trivial, the technique is general, and it works on real-world problems. See [Chapter 5, State, on page 113](#) for more on concurrency and STM in Clojure.

Clojure Embraces the Java Virtual Machine

Clojure gives you clean, simple, direct access to Java. You can call any Java API directly:

```
(System/getProperties)
-> {java.runtime.name=Java(TM) SE Runtime Environment
... many more ...}
```

Clojure adds a lot of syntactic sugar for calling Java. We won't get into the details here (see [Section 2.5, Calling Java, on page 43](#)), but notice that in the following code the Clojure version has both fewer dots *and fewer parentheses* than the Java version:

```
// Java
"hello".getClass().getProtectionDomain()

; Clojure
(.. "hello" getClass getProtectionDomain)
```

Clojure provides simple functions for implementing Java interfaces and subclassing Java classes. Also, Clojure functions all implement `Callable` and `Runnable`. This makes it trivial to pass the following anonymous function to the constructor for a Java Thread.

```
(.start (new Thread (fn [] (println "Hello" (Thread/currentThread)))))
-> Hello #<Thread Thread[Thread-0,5,main]>
```

The funny output here is Clojure's way of printing a Java instance. `Thread` is the class name of the instance, and `Thread[Thread-0,5,main]` is the instance's `toString` representation.

(Note that in the preceding example the new thread will run to completion, but its output may interleave in some strange way with the REPL prompt. This is not a problem with Clojure but simply the result of having more than one thread writing to an output stream.)

Because the Java invocation syntax in Clojure is clean and simple, it is idiomatic to use Java directly, rather than to hide Java behind Lispy wrappers.

Now that you have seen a few of the reasons to use Clojure, it is time to start writing some code.

1.2 Clojure Coding Quick Start

To run Clojure and the code in this book, you need two things:

- *A Java runtime.* Download⁴ and install Java version 5 or greater. Java version 6 has significant performance improvements and better exception reporting, so prefer this if possible.
- *Leiningen.*⁵ Leiningen is a tool for managing dependencies and launching tasks against your code. It is also the most common tool for this job in the Clojure space.

You will use Leiningen to install Clojure and all of the dependencies for the sample code in this book. If you already have Leiningen installed, you should be familiar with the basics. If not, you should take a quick tour of Leiningen's GitHub page,⁶ where you will find install instructions as well as basic usage instructions. Don't worry about learning everything now, though, because this book will guide you through the commands necessary to follow along at home.

While you are working through the book, use the version of Clojure tied to the book's sample code. After you read the book, you can follow the instructions in [Building Clojure Yourself, on page 12](#) to build an up-to-the-minute version of Clojure.

See [Section 6, Downloading Sample Code, on page xxii](#) for instructions on downloading the sample code. Once you have downloaded the sample code, you will need to use Leiningen to fetch the dependencies. From the root of the example code folder, run this:

```
lein deps
```

4. <http://www.oracle.com/technetwork/java/javase/downloads/index.html>

5. <http://github.com/technomancy/leiningen>

6. <http://github.com/technomancy/leiningen>

Building Clojure Yourself

You may want to build Clojure from source to get access to newer features and bug fixes. Here's how:

```
git clone git://github.com/clojure/clojure.git
cd clojure
mvn package
```

The sample code is regularly updated to match the current development head of Clojure. Check the README file in the sample code to see the revision numbers that the samples were most recently tested with.

The dependencies will be downloaded and placed in the proper location. You can test your install by navigating to the directory where you placed the sample code and running a Clojure *read-eval-print loop* (REPL). Leiningen contains a REPL launch script that loads Clojure along with the dependencies that we will need later in the book.

```
lein repl
```

When you successfully launch the REPL, it should prompt you with `user=>`:

```
Clojure
user=>
```

Now you are ready for “Hello World.”

Using the REPL

To see how to use the REPL, let's create a few variants of “Hello World.” First, type `(println "hello world")` at the REPL prompt:

```
user=> (println "hello world")
-> hello world
```

The second line, `hello world`, is the console output you requested.

Next, encapsulate your “Hello World” into a function that can address a person by name:

```
(defn hello [name] (str "Hello, " name))
-> #'user/hello
```

Let's break this down:

- `defn` defines a function.
- `hello` is the function name.
- `hello` takes one argument, `name`.

- `str` is a function call that concatenates an arbitrary list of arguments into a string.
- `defn`, `hello`, `name`, and `str` are all *symbols*, which are names that refer to things. Legal symbols are defined in [Symbols, on page 25](#).

Look at the return value, `#'user/hello`. The prefix `#'` indicates that the function was stored in a Clojure *var*, and `user` is the *namespace* of the function. (The `user` namespace is the REPL default, like the default package in Java.) You do not need to worry about vars and namespaces yet; they are covered in [Section 2.4, Vars, Bindings, and Namespaces, on page 36](#).

Now you can call `hello`, passing in your name:

```
user=> (hello "Stu")
-> "Hello, Stu"
```

If you get your REPL into a state that confuses you, the simplest fix is to kill the REPL with `CTRL+C` on Windows or `CTRL+D` on *nix and then start another one.

Special Variables

The REPL includes several useful special variables. When you are working in the REPL, the results of evaluating the three most recent expressions are stored in the special variables `*1`, `*2`, and `*3`, respectively. This makes it easy to work iteratively. Say hello to a few different names:

```
user=> (hello "Stu")
-> "Hello, Stu"

user=> (hello "Clojure")
-> "Hello, Clojure"
```

Now, you can use the special variables to combine the results of your recent work:

```
(str *1 " and " *2)
-> "Hello, Clojure and Hello, Stu"
```

If you make a mistake in the REPL, you will see a Java exception. The details are often omitted for brevity. For example, dividing by zero is a no-no:

```
user=> (/ 1 0)
-> ArithmeticException Divide by zero  clojure.lang.Numbers.divide
```

Here the problem is obvious, but sometimes the problem is more subtle and you want the detailed stack trace. The `*e` special variable holds the last

exception. Because Clojure exceptions are Java exceptions, you can ask for the stacktrace by calling `pst` (print stacktrace).⁷

```
user=> (pst)
-> ArithmeticException Divide by zero
|   clojure.lang.Numbers.divide
|   sun.reflect.NativeMethodAccessorImpl.invoke0
|   sun.reflect.NativeMethodAccessorImpl.invoke
|   sun.reflect.DelegatingMethodAccessorImpl.invoke
|   java.lang.reflect.Method.invoke
|   clojure.lang.Reflector.invokeMatchingMethod
|   clojure.lang.Reflector.invokeStaticMethod
|   user/eval1677
|   clojure.lang.Compiler.eval
|   clojure.lang.Compiler.eval
|   clojure.core/eval
```

Java interop is covered in [Chapter 9, Java Down and Dirty, on page 203](#).

If you have a block of code that is too large to conveniently type at the REPL, save the code into a file, and then load that file from the REPL. You can use an absolute path or a path relative to where you launched the REPL:

```
; save some work in temp.clj, and then ...
user=> (load-file "temp.clj")
```

The REPL is a terrific environment for trying ideas and getting immediate feedback. For best results, keep a REPL open at all times while reading this book.

Adding Shared State

The `hello` function of the previous section is *pure*; that is, it has no side effects. Pure functions are easy to develop, test, and understand, and you should prefer them for many tasks.

That said, most programs have some shared state and will use impure functions to manage that shared state. Let's extend `hello` to keep track of past visitors. First, you will need a data structure to track the visitors. A set will do the trick:

```
#{}
-> #{} 
```

The `#{}` is a literal for an empty set. Next, you will need `conj`:

```
(conj coll item)
```

7. `pst` is available only in Clojure 1.3.0 and greater.

conj is short for conjoin, and it builds a new collection with an item added. conj an element onto a set to see that a new set is created:

```
(conj #{} "Stu")
-> #{"Stu"}
```

Now that you can build new sets, you need some way to keep track of the *current* set of visitors. Clojure provides several *reference types* (refs) for this purpose. The most basic reference type is the atom:

```
(atom initial-state)
```

To name your atom, you can use def:

```
(def symbol initial-value?)
```

def is like defn but more general. A def can define functions *or* data. Use atom to create an atom, and use def to bind the atom to the name visitors:

```
(def visitors (atom #{}))
-> #'user/visitors
```

To update a reference, you must use a function such as swap!:

```
(swap! r update-fn & args)
```

swap! applies an update-fn to reference r, with optional args if necessary. Try to swap! a visitor into visitors, using conj as the update function:

```
(swap! visitors conj "Stu")
-> #{"Stu"}
```

atom is one of several reference types in Clojure. Choosing the appropriate reference type requires care (discussed in [Chapter 5, State, on page 113](#)).

At any time, you can peek inside the ref with deref or with the shorter @:

```
(deref visitors)
-> #{"Stu"}
```

```
@visitors
-> #{"Stu"}
```

Now you are ready to build the new, more elaborate version of hello:

```
src/examples/introduction.clj
```

```
(defn hello
  "Writes hello message to *out*. Calls you by username.
  Knows if you have been here before."
  [username]
  (swap! visitors conj username)
  (str "Hello, " username))
```

Next, check that visitors are correctly tracked in memory:

```
(hello "Rich")
-> "Hello, Rich"

@visitors
-> #{ "Aaron" "Stu" "Rich" }
```

In all probability, your visitors list is different from the one shown here. That's the problem with state! Your results will vary, depending on when things happened. You can reason about a function with direct local knowledge. Reasoning about state requires a full understanding of history.

Avoid state where possible. But when you need it, make it sane and manageable by using refs such as atoms. Atoms (and all other Clojure reference types) are safe for multiple threads and processors. Better yet, this safety comes without any need for locks, which are notoriously tricky to use.

At this point, you should feel comfortable entering small bits of code at the REPL. Larger units of code aren't that different; you can load and run Clojure libraries from the REPL as well. Let's explore that next.

1.3 Exploring Clojure Libraries

Clojure code is packaged in *libraries*. Each Clojure library belongs to a *namespace*, which is analogous to a Java package. You can load a Clojure library with `require`:

```
(require quoted-namespace-symbol)
```

When you require a library named `clojure.java.io`, Clojure looks for a file named `clojure/java/io.clj` on the CLASSPATH. Try it:

```
user=> (require 'clojure.java.io)
-> nil
```

The leading single quote (`'`) is required, and it *quotes* the library name (quoting is covered in [Section 2.2, Reader Macros, on page 30](#)). The `nil` returned indicates success. While you are at it, test that you can load the sample code for this chapter, `examples.introduction`:

```
user=> (require 'examples.introduction)
-> nil
```

The `examples.introduction` library includes an implementation of the Fibonacci numbers, which is the traditional “Hello World” program for functional languages. We will explore the Fibonacci numbers in more detail in [Section 4.2](#),

[How to Be Lazy, on page 90](#). For now, just make sure that you can execute the sample function `fib`s. Enter the following line of code at the REPL to take the first ten Fibonacci numbers:

```
(take 10 examples.introduction/fibs)
-> (0 1 1 2 3 5 8 13 21 34)
```

If you see the first ten Fibonacci numbers as listed here, you have successfully installed the book samples.

The book samples are all unit tested, with tests located in the `examples/test` directory. The tests for the samples themselves are not explicitly covered in the book, but you may find them useful for reference. You can run the unit tests yourself with `lein test`.

Require and Use

When you require a Clojure library, you must refer to items in the library with a namespace-qualified name. Instead of `fib`s, you must say `examples.introduction/fibs`. Make sure to launch a new REPL,⁸ and then try it:

```
(require 'examples.introduction)
-> nil

(take 10 examples.introduction/fibs)
-> (0 1 1 2 3 5 8 13 21 34)
```

Fully qualified names get old quickly. You can refer a namespace, creating mappings for all its names in your current namespace:

```
(refer quoted-namespace-symbol)
```

Call `refer` on `examples.introduction`, and verify that you can then call `fib`s directly:

```
(refer 'examples.introduction)
-> nil

(take 10 fibs)
-> (0 1 1 2 3 5 8 13 21 34)
```

For convenience, the `use` function will require and refer a library in a single step:

```
(use quoted-namespace-symbol)
```

From a new REPL you should be able to do the following:

8. Creating a new REPL will prevent name collisions between your previous work and the sample code functions of the same name. This is not a problem in real-world development, as you will see in [Namespaces, on page 40](#).

```
(use 'examples.introduction)
-> nil
```

```
(take 10 fibs)
-> (0 1 1 2 3 5 8 13 21 34)
```

As you are working through the book samples, you can call `require` or `use` with a `:reload` flag to force a library to reload:

```
(use :reload 'examples.introduction)
-> nil
```

The `:reload` flag is useful if you are making changes and want to see results without restarting the REPL.

Finding Documentation

Often you can find the documentation you need right at the REPL. The most basic helper function⁹ is `doc`:

```
(doc name)
```

Use `doc` to print the documentation for `str`:

```
user=> (doc str)
-----
clojure.core/str
([] [x] [x & ys])
With no args, returns the empty string. With one arg x, returns
x.toString(). (str nil) returns the empty string. With more than
one arg, returns the concatenation of the str values of the args.
```

The first line of `doc`'s output contains the fully qualified name of the function. The next line contains the possible argument lists, generated directly from the code. (Some common argument names and their uses are explained in [Conventions for Parameter Names, on page 19](#).) Finally, the remaining lines contain the function's *doc string*, if the function definition included one.

You can add a doc string to your own functions by placing it immediately after the function name:

```
src/examples/introduction.clj
(defn hello
  "Writes hello message to *out*. Calls you by username"
  [username]
  (println (str "Hello, " username)))
```

9. `doc` is actually a Clojure macro.

Conventions for Parameter Names

The documentation strings for `reduce` and `areduce` show several terse parameter names. Here are some parameter names and how they are normally used:

Parameter	Usage
<code>a</code>	A Java array
<code>agt</code>	An agent
<code>coll</code>	A collection
<code>expr</code>	An expression
<code>f</code>	A function
<code>idx</code>	Index
<code>r</code>	A ref
<code>v</code>	A vector
<code>val</code>	A value

These names may seem a little terse, but there is a good reason for them: the “good names” are often taken by Clojure functions! Naming a parameter that collides with a function name is legal but considered bad style: the parameter will shadow the function, which will be unavailable while the parameter is in scope. So, don’t call your refs `ref`, your agents `agent`, or your counts `count`. Those names refer to functions.

Sometimes you will not know the exact name you want documentation for. The `find-doc` function will search for anything whose doc output matches a regular expression or string you pass in:

```
(find-doc s)
```

Use `find-doc` to explore how Clojure does `reduce`:

```
user=> (find-doc "reduce")
-----
clojure/areduce
([a idx ret init expr])
Macro
... details elided ...
-----
clojure/reduce
([f coll] [f val coll])
... details elided ...
```

`reduce` reduces Clojure collections and is covered in [Transforming Sequences, on page 66](#). `areduce` is for interoperability with Java arrays and is covered in [Using Java Collections, on page 216](#).

Much of Clojure is written in Clojure, and it is instructive to read the source code. You can view the source of a Clojure function using the `repl` library.

```
(clojure.repl/source a-symbol)
```

Try viewing the source of the simple identity function:

```
(use 'clojure.repl)
(source identity)

-> (defn identity
    "Returns its argument."
    {:added "1.0"
     :static true}
    [x] x)
```

Of course, you can also use Java's Reflection API. You can use methods such as `class`, `ancestors`, and `instance?` to reflect against the underlying Java object model and tell, for example, that Clojure's collections are also Java collections:

```
(ancestors (class [1 2 3]))

-> #{clojure.lang.ILookup clojure.lang.Sequential
    java.lang.Object clojure.lang.Indexed
    java.lang.Iterable clojure.lang.IObj
    clojure.lang.IPersistentCollection
    clojure.lang.IPersistentVector clojure.lang.AFn
    java.lang.Comparable java.util.RandomAccess
    clojure.lang.Associative
    clojure.lang.APersistentVector clojure.lang.Counted
    clojure.lang.Reversible clojure.lang.IPersistentStack
    java.util.List clojure.lang.IEditableCollection
    clojure.lang.IFn clojure.lang.Seqable
    java.util.Collection java.util.concurrent.Callable
    clojure.lang.IMeta java.io.Serializable java.lang.Runnable}
```

Clojure's complete API is documented online at <http://clojure.github.com/clojure>. The right sidebar links to all functions and macros by name, and the left sidebar links to a set of overview articles on various Clojure features.

1.4 Wrapping Up

You have just gotten the whirlwind tour of Clojure. You have seen Clojure's expressive syntax, learned about Clojure's approach to Lisp, and seen how easy it is to call Java code from Clojure.

You have Clojure running in your own environment, and you have written short programs at the REPL to demonstrate functional programming and the reference model for dealing with state. Now it is time to explore the entire language.